{EPITECH}

MY_SUDO_

< SUBSTITUTE USER DO />

MY_SUDO

You are asked to re-implement the `sudo` command.

> With great power comes great responsability.
>
> **— Ben Parker —**

Binary name: my_sudo
Language: C
Compilation: via Makefile, including re, clean and fclean rules
Authorized functions: All the libC minus forbidden functions.
Forbidden functions: getpw*, *getsp*, getgr*, *fork*, *clone*, openat*, getpass, getlogin*, cuserid, system

- ✓ The totality of your source files, except all useless files (binary, temp files, objfiles,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.
- ✓ Error messages have to be written on the error output.
- ✓ Exit status 0 in case of success.
- ✓ Exit status 84 in case of failure.
- ✓ Makefile should not change the ownership of the binary, adding `chown root:root my_sudo` will result in a compilation failure.
- ✓ Makefile should not create a SUID binary, adding `chmod +s my_sudo` will result in a compilation failure.

{EPITECH}

Your program must be able to be invoqued as such:

```
Terminal
$>  ./my_sudo -h
usage: ./my_sudo -h
usage: ./my_sudo [-ug] [command [args ...]]
```

Flags:

-h,
Display a short help message to the standard output and exit.

-u user,
Run the command as a user other than the default target user (usually root)

-g group,
Run the command with the primary group set to group instead of the primary group specified by the target user's password database entry

User authentication

You must be able to authenticate users invoquing the my_sudo command with their password.

> `man shadow`, `man crypt`

In case of an incorrect password you should allow the user of your program to make two additional attempts.

> Unlike the original sudo command, the password will be read directly from stdin and not via a pty. Although a bad security practice, this is a technical requirement in order to pass the automated tests.

```
$>  (echo "good_password") | ./my_sudo printf "Hello world"
Hello world
$> whoami
current_user
$> (echo "bad_password"; sleep 1; echo "good_password") | ./my_sudo printf "Hello world"
[my_sudo] password for current_user: Sorry, try again.
[my_sudo] password for current_user: Hello world
```

Outside of the scope of this project:

- ✓ The usage of PAM (Pluggable Authentication Module) library to handle user authentication.
- ✓ Credential caching

Sudoers

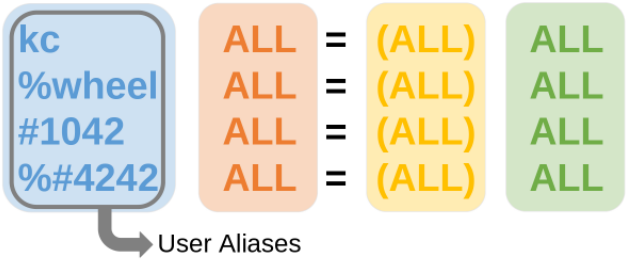
Your program must read its security policy from `/etc/sudoers`.

```
man sudoers
```

Example:

Considering the following sudoers file:

```
$>  cat /etc/sudoers
...
kc ALL=(ALL) ALL
%wheel ALL=(ALL) ALL
#1042 ALL=(ALL) ALL
%#4242 ALL=(ALL) ALL
...
```



For this project you must handle only User Aliases in the sudoers file.
(ie: Always consider that everything after the User Alias is set to "ALL=(ALL) ALL" for the autograder)
However, being able to handle the entire User Specification would be a nice bonus.

```
Terminal
$>  whoami
kc
$> ./my_sudo /usr/bin/whoami
[my_sudo] password for kc:
root
```

```
Terminal
$>  whoami
kc
$> ./my_sudo -u toto whoami
[my_sudo] password for kc:
toto
$> ls -l top_secret
total 4
-rw-r-----. 1 toto admins 3 Jun 27 12:09 top_secret
$> cat top_secret
cat: top_secret: Permission denied
$> ./my_sudo -u toto cat top_secret
42
```

Don't hesitate to create some dummy users and groups on your computer in order to toroughly test your program.
Your /etc/sudoers, /etc/passwd, /etc/shadow, /etc/group files are used by the system, be careful.
Having a backup mechanism might preserve your system.
Could be file backups, activated root account, working in a virtual machine or a container.

You may also want to modify your `/etc/sudoers` file, but be careful to not break your GNU/Linux installation. Try to find a safe way to edit this file.

More examples

Using the same `/etc/sudoers` file.

```
Terminal
$>  whoami
kc
$> groups toto
toto : admins docker
$> ./my_sudo -u toto yes
[my_sudo] password for kc:
$> ps -C yes -o ruser,rgroup,euser,egroup
COMMAND RUSER RGROUP EUSER EGROUP
yes toto admins toto admins
$> ./my_sudo -u toto -g docker yes
[my_sudo] password for kc:
$> ps -C yes -o ruser,rgroup,euser,egroup
COMMAND RUSER RGROUP EUSER EGROUP
yes toto docker toto docker
```

```
Terminal
$>  whoami
tata
$> groups tata
tata : tata audio cdrom video wheel
$> ./my_sudo whoami
[my_sudo] password for tata:
root
```

```
Terminal
$>  whoami
tutu
$> groups tutu
tutu : tutu audio cdrom video
$> ./my_sudo whoami
[my_sudo] password for tutu:
tutu is not in the my_sudoers file.
```

Terminal

```
$>  id
uid=1009(titi) gid=1009(titi) groups=1009(titi),1042(users),4242(birds)
$> ./my_sudo whoami
[my_sudo] password for titi:
root
```

Terminal

```
$>  whoami
kc
$> env | grep -E "^SHELL="
SHELL=/bin/bash
$> ./my_sudo -s
[my_sudo] password for kc:
$> ps -p $$ -o cmd,user,group
CMD USER GROUP
/bin/bash root root
$> exit
$> whoami
kc
$> export SHELL=/bin/sh
$> ./my_sudo -s
[my_sudo] password for kc:
$> ps -p $$ -o cmd,user,group
CMD USER GROUP
/bin/sh root root
$> exit
$> whoami
kc
$> unset SHELL
$> grep -E "^kc|^toto" /etc/passwd
kc:x:1000:1000:KC:/home/kc:/bin/zsh
toto:x:1001:4200::/home/toto:/bin/bash
$> ./my_sudo -s -u toto
[my_sudo] password for kc:
$> ps -p $$ -o cmd,user,group
CMD USER GROUP
/bin/zsh toto admins
```

Hint

Type, understand, reproduce

```
Terminal
$>  ps -o 'pid,ruser,ruid,rgid,euser,euid,egid,suid,sgid,cmd'
...
$> sudo ps -o 'pid,ruser,ruid,rgid,euser,euid,egid,suid,sgid,cmd' | tail -n -1
```

Security

Your program should not disclose information on the user credentials upon password entering.
Remember that user input should never be trusted (password input, env variables)

Bonus

Here are some bonus ideas:

- ✓ `-l`, `--list`
- ✓ `-E`, `--preserv-env`
- ✓ `-s`, `--shell`
- ✓ Credentials Caching
- ✓ Logging

v 1.1

{EPITECH}